PROIECT · PARTEA 1

Modelarea unei funcții necunoscute

Aproximare polinomială în două variabile



ID grup: [NUMĂRUL UNIC]

Membri: [NUME 1] · [NUME 2] · [NUME 3]

Prezentare — maximum 6 minute

Problema și datele disponibile

Observăm o funcție statică, neliniară, afectată de zgomot aditiv cu medie zero.

IDENTIFICARE · id

X = {X□, X□}

Y(i,j) este ieșirea observată în punctul (X□(i), X□(j)).

date de antrenare

VALIDARE · val

aceeași structură X, Y

Set separat: verifică dacă modelul generalizează la date nefolosite la estimarea parametrilor.

alegerea gradului

Scop: eroare mică pe validare, fără a confunda zgomotul cu forma funcției.

Modelul polinomial în două variabile

Pentru grad maxim m folosim termenii $x_1^p x_2^q$ cu $p + q \le m$.

$$\hat{y}(x_1,x_2) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 + \dots$$

$$\hat{y}(x_1,x_2) = \varphi(x_1,x_2) \cdot \theta$$

Neliniar în intrări, liniar în parametrii θ.

Număr de termeni

$$(m + 1)(m + 2) / 2$$

m = 1 → 3 coeficienți
m = 2 → 6 coeficienți
m = 3 → 10 coeficienți

Ordinea termenilor în coloane poate varia, dacă rămâne aceeași la antrenare și predicție.

Cum estimăm și validăm parametrii

1

Construim Φ

O linie pentru fiecare punct; coloanele conțin termenii polinomului.

→

2

Estimăm θ

Rezolvăm Φ θ ≈ y cu operatorul MATLAB \.

→

3

Calculăm MSE

Media pătratelor diferențelor între valori reale și estimate.

→

4

Validăm separat

Aplicăm θ estimat pe id pentru a prezice datele din val.

MSE = media [(valoare reală − valoare estimată)²]

Alegerea gradului m

Antrenăm câte un model pentru fiecare grad; criteriul este MSE pe validare.

m mic	**Model prea rigid**	MSE validare poate rămâne mare
m potrivit	**Surprinde forma funcției**	MSE validare tinde să fie minim
m mare	**Mai multă flexibilitate**	Poate învăța și zgomotul (overfitting)

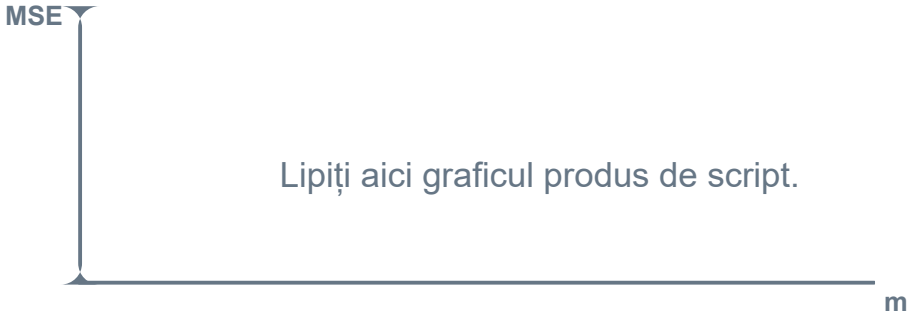
$$m^* = \arg\min_m \mathrm{MSE_val}(m)$$

După alegere, raportăm MSE pe ambele seturi și inspectăm graficele.

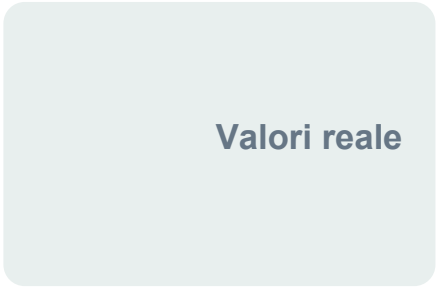
Rezultatele experimentale

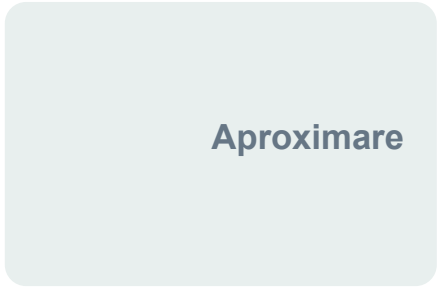
Completați după rularea scriptului pe fișierul real al grupei.

MSE în funcție de gradul m



Suprafața: reală vs aproximată





Înlocuiți cu graficele reale pentru gradul m*.

Discuție și concluzie

- Modelul folosește o bază polinomială completă de grad m pentru două intrări.
- Parametrii se estimează pe id; setul val rămâne pentru evaluare.
- Gradul optim se alege după minimul MSE de validare, nu după eroarea de antrenare.

CONCLUZIE

Modelul ales echilibrează fidelitatea aproximării și generalizarea.

Anexă · cod MATLAB (1/3)

```
%% Aproximarea unei functii cu doua intrari folosind polinoame
% Scriptul presupune ca fisierul date.mat contine structurile id si val.
% Fiecare structura trebuie sa aiba campurile X (cell cu doi vectori) si Y.
% Daca fisierul are alt nume, modificati numele din linia de mai jos.

clear;
clc;
close all;

load('date.mat', 'id', 'val');

gradeDeIncercat = 1:5;
numarGrade = numel(gradeDeIncercat);
eroareAntrenare = zeros(1, numarGrade);
eroareValidare = zeros(1, numarGrade);
modele = cell(1, numarGrade);

% Convertim grilele de intrare in liste de puncte si tinte.
[puncteAntrenare, valoriAntrenare] = pregatesteDate(id);
[puncteValidare, valoriValidare] = pregatesteDate(val);

for indexGrad = 1:numarGrade
gradCurent = gradeDeIncercat(indexGrad);

% Fiecare linie a matricei contine termenii polinomului pentru un punct.
matriceAntrenare = construiesteMatricePolinomiala(puncteAntrenare, gradCurent);
matriceValidare = construiesteMatricePolinomiala(puncteValidare, gradCurent);

% A_x0008_ gaseste coeficientii care minimizeaza suma patratelor erorilor.
coeficienti = matriceAntrenare \ valoriAntrenare;
predictiiAntrenare = matriceAntrenare * coeficienti;
predictiiValidare = matriceValidare * coeficienti;

eroareAntrenare(indexGrad) = mean((predictiiAntrenare - valoriAntrenare).^2);
eroareValidare(indexGrad) = mean((predictiiValidare - valoriValidare).^2);

modele{indexGrad} = struct('grad', gradCurent, 'coeficienti', coeficienti);
end
```

Anexă · cod MATLAB (2/3)

```
% Afisam erorile pentru a compara gradele si a observa posibila supraantrenare.
   disp(table(gradeDeIncercat(:), eroareAntrenare(:), eroareValidare(:), ...
       'VariableNames', {'Grad', 'EroareAntrenare', 'EroareValidare'}));

                  [~, indexCelMaiBun] = min(eroareValidare);
                     modelAles = modele{indexCelMaiBun};
     fprintf('Gradul ales dupa eroarea de validare: %d\n', modelAles.grad);

                   figure('Name', 'Comparatia erorilor');
        plot(gradeDeIncercat, eroareAntrenare, '-o', 'LineWidth', 1.5);
                                   hold on;
         plot(gradeDeIncercat, eroareValidare, '-s', 'LineWidth', 1.5);
                      xlabel('Gradul polinomului');
                    ylabel('Eroarea medie patratica');
              legend('Antrenare', 'Validare', 'Location', 'best');
                    title('Alegerea gradului polinomului');
                                   grid on;

     % Reprezentam predictiile si valorile masurate pe grila fiecarui set.
                reprezintaRezultate(id, modelAles, 'Antrenare');
                reprezintaRezultate(val, modelAles, 'Validare');

                              %% Functii locale
            function [puncte, valori] = pregatesteDate(setDate)
                     vectorIntrare1 = setDate.X{1}(:);
                     vectorIntrare2 = setDate.X{2}(:);
    [grilaIntrare1, grilaIntrare2] = ndgrid(vectorIntrare1, vectorIntrare2);

                puncte = [grilaIntrare1(:), grilaIntrare2(:)];
                           valori = setDate.Y(:);

                       if numel(valori) ~= size(puncte, 1)
   error('Dimensiunile lui Y nu corespund grilei formate din X{1} si X{2}.');
                                      end
                                     end

function matricePolinomiala = construiesteMatricePolinomiala(puncte, gradMaxim)
```

Anexă · cod MATLAB (3/3)

```
                intrare1 = puncte(:, 1);
                intrare2 = puncte(:, 2);
        matricePolinomiala = ones(size(puncte, 1), 1);

    % Adaugam puterile pure, apoi termenii amestecati pentru fiecare grad.
                for gradTermen = 1:gradMaxim
            matricePolinomiala(:, end + 1) = intrare1.^gradTermen;
            matricePolinomiala(:, end + 1) = intrare2.^gradTermen;

                    for putereIntrare1 = gradTermen-1:-1:1
                putereIntrare2 = gradTermen - putereIntrare1;
                    matricePolinomiala(:, end + 1) = ...
            intrare1.^putereIntrare1 .* intrare2.^putereIntrare2;
                                end
                                end
                               end

            function reprezintaRezultate(setDate, model, numeSet)
               [puncte, valoriReale] = pregatesteDate(setDate);
     matricePolinomiala = construiesteMatricePolinomiala(puncte, model.grad);
            valoriEstimate = matricePolinomiala * model.coeficienti;

   valoriReale = reshape(valoriReale, numel(setDate.X{1}), numel(setDate.X{2}));
valoriEstimate = reshape(valoriEstimate, numel(setDate.X{1}), numel(setDate.X{2}));
    [grilaIntrare1, grilaIntrare2] = ndgrid(setDate.X{1}(:), setDate.X{2}(:));

                figure('Name', ['Rezultate - ' numeSet]);
                            subplot(1, 2, 1);
              surf(grilaIntrare1, grilaIntrare2, valoriReale);
                    title(['Valori reale - ' numeSet]);
             xlabel('x_1'); ylabel('x_2'); zlabel('f(x_1,x_2)');
                             shading interp;

                            subplot(1, 2, 2);
             surf(grilaIntrare1, grilaIntrare2, valoriEstimate);
                title(['Aproximare polinomiala - ' numeSet]);
             xlabel('x_1'); ylabel('x_2'); zlabel('g(x_1,x_2)');
                             shading interp;
                                  end
```